Plataforma educativa con IA para Argentina: documento de la idea

Oct 8, 2026 · @dario anzaudo

Resumen y visión

La idea es una plataforma educativa para primaria y secundaria en Argentina que combina tres capas: un **tutor con IA adaptado a los intereses de cada alumno**, **grupos y subgrupos** que el docente organiza por nivel e intereses, y **proyectos interdisciplinarios** que pueden crecer hacia un impacto real (emprendimiento, solución de un problema comunitario, financiamiento).

El tutor es el punto de partida, pero no el diferencial: apps como Khanmigo ya lo resuelven. Lo que distingue a esta propuesta es que el aprendizaje se organiza alrededor de **proyectos anclados al currículo oficial**, para que las escuelas puedan adoptarla sin salirse de lo que ya tienen que enseñar.

Principios de diseño

- No reemplaza al docente: le da herramientas y datos.
- No da respuestas hechas: guía con preguntas (método socrático).
- No se aleja del currículo: cada proyecto se mapea a contenidos oficiales.
- No toca la evaluación oficial: genera evidencia, la calificación sigue siendo del docente.

Diagnóstico y oportunidad

Las tareas escolares y los métodos de estudio en la mesa de la cocina casi no cambiaron en cuatro generaciones, y la IA todavía no entró en esa rutina. Según la charla de partida, entre el 75% y el 80% de las personas aún no incorporó herramientas de IA por temor, falta de capacitación o resistencia al cambio (cifra de la conversación con Gemini, pendiente de verificar con fuente propia).

En Argentina y Latinoamérica los modelos generales (Gemini, ChatGPT) se usan de forma cotidiana e informal, pero las aplicaciones pedagógicas específicas están en una etapa temprana o de nicho. Ese es el espacio para una solución adaptada a la región, al currículo local y a la realidad de las escuelas.

Las claves pedagógicas (Santiago Bilinkis)

- **Personalización por intereses:** explicar temas áridos con metáforas del deporte, los videojuegos o la cocina que le gustan al alumno.

- **Aprendizaje conversacional:** dialogar con personajes históricos o científicos para que el contenido sea cercano.
- **Enfoque socrático:** la herramienta no da la respuesta, hace preguntas que verifican si el chico entendió.
- **Aprender a preguntar:** desarrollar la habilidad de formular buenas preguntas.

Alternativas existentes y diferencial

Hay tutores de IA y plataformas para docentes, pero ninguno combina grupos, proyectos y currículo argentino.

Herramienta	Qué hace	Qué le falta a esta idea
Khanmigo (Khan Academy)	Tutor socrático que guía paso a paso sin dar la solución	Proyectos, grupos por intereses, currículo local
Hello History	Conversar en español con personajes históricos y científicos	Estructura de aula y seguimiento
Synthesis Tutor	Lecciones interactivas de matemática y lógica con dificultad adaptativa	Integración de materias, proyectos
Mizou / SchoolAI	Docentes y familias crean asistentes ajustados a un tema	Escalera de proyectos hacia impacto real

Conceptos que surgieron en la charla inicial y siguen vigentes como funciones: tutor adaptativo por interés ("A tu manera"), escáner que transforma la guía impresa del colegio en minijuegos ("Mesa de la Cocina 2.0") y co-piloto para padres que detecta en qué paso se traba el alumno. Se incorporan como módulos posteriores al núcleo.

Usuarios y roles

La plataforma tiene tres roles, con un punto de entrada claro: **el docente y el aula**.

Rol	Función	Por qué importa
Alumno	Aprende con el tutor, trabaja en proyectos y arma su portfolio	Es quien más usa la app

Rol	Función	Por qué importa
Docente	Crea grupos y proyectos, ve el progreso y dónde se traba cada chico	Es el canal de distribución: los grupos ya existen y llega a ~30 alumnos de una vez
Familia	Acompaña, ve el progreso, recibe sugerencias para ayudar sin frustrarse	A veces paga; da el consentimiento para menores

Entrar por el docente evita el problema de arranque en frío, porque los cursos ya están armados. La vista de familia es secundaria en la primera versión.

Grupos y subgrupos

La app maneja dos tipos de grupo, y el docente siempre tiene la última palabra.

- **Formales:** escuela → curso → división. Los crea el docente o la institución.
- **Dinámicos:** se arman por afinidad. Criterios posibles: interés (fútbol, robótica, cocina), nivel en una materia, objetivo (un proyecto) o complementariedad (quien domina matemática con quien domina diseño).

El **nivel se mide por materia o habilidad, no por año escolar**: un chico de 2º año puede estar en nivel de 4º en matemática y de 1º en lectura. Eso permite subgrupos reales.

La app **sugiere** agrupaciones y el docente las aprueba. En primaria casi siempre los arma el docente; en secundaria los alumnos pueden tener más autonomía para formarlos por proyecto.

Proyectos y escalera de impacto

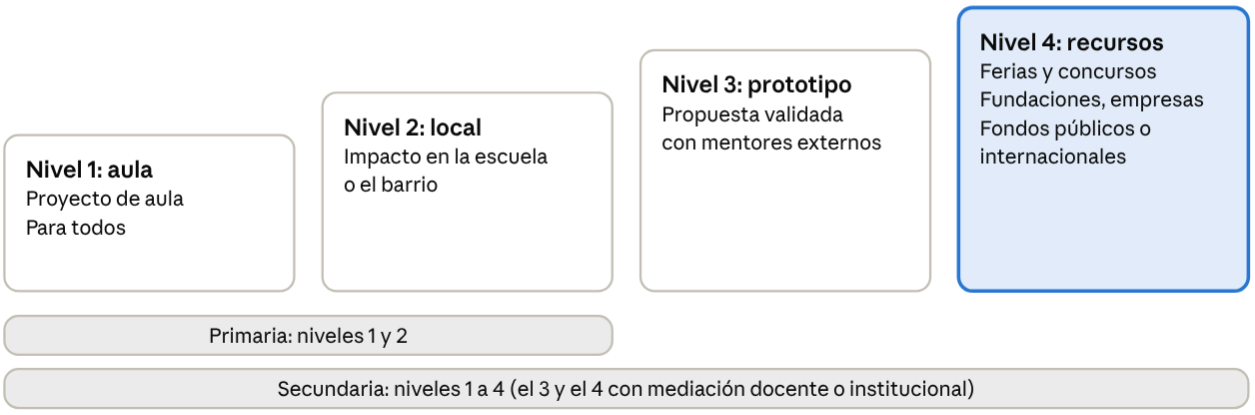
Los proyectos son el eje de la plataforma: integran materias, generan evidencia y pueden crecer hacia algo con impacto real.

Cómo funciona un proyecto

1. Alguien define el proyecto (por ejemplo, reducir el desperdicio de comida en el comedor de la escuela).
2. La app lo descompone en saberes necesarios: matemática para medir, ciencias naturales, lengua para comunicar, economía para costear.
3. Cada saber se convierte en misiones de aprendizaje con el tutor, adaptadas al interés del alumno.
4. Se asignan roles dentro del grupo.
5. Se arma un portfolio con la evidencia.

Si los proyectos se mapean a los NAP, el docente puede justificar ante la dirección que cubren contenidos curriculares.

Los proyectos escalan de la clase al financiamiento en cuatro niveles



escalera de proyectos · 4 niveles y alcance por nivel educativo

No todos los proyectos recorren la escalera completa. Primaria llega hasta el nivel 2; los niveles 3 y 4 se reservan a secundaria, siempre con mediación docente o institucional. En el nivel 4 la app podría conectar proyectos con quienes pueden apoyarlos (mentores, empresas, ONG), pero ese matching queda para después del MVP.

Pedagogía del tutor

El tutor combina cuatro ideas que se refuerzan entre sí.

1. **Personalización por interés:** el alumno elige su pasión y el tutor explica el tema con analogías de ese mundo.
2. **Conversación:** el alumno puede "hablar" con personajes históricos o científicos para que el contenido resulte cercano.
3. **Socrático, anti copia y pega:** el tutor nunca entrega la respuesta hecha; hace preguntas orientadoras y verifica la comprensión con un cuestionario.
4. **Aprender a preguntar:** se evalúa y se entrena la calidad de las preguntas del alumno, no solo sus respuestas.

El tutor recibe como contexto el contenido curricular y el año del alumno, para explicar al nivel correcto. Dentro de un proyecto, se activa como **misión de aprendizaje just-in-time**: aparece cuando el proyecto requiere un saber que el alumno todavía no tiene.

Capa curricular y marco normativo

La plataforma se apoya en el currículo oficial para reducir la resistencia de las instituciones al cambio. Hay dos capas normativas que estudiar:

- **Nacional:** Ley de Educación Nacional 26.206, acuerdos del Consejo Federal de Educación y los NAP (Núcleos de Aprendizajes Prioritarios), el piso común.
- **Provincial:** el diseño curricular de cada jurisdicción, su régimen académico (cómo se evalúa, califica y promociona) y su calendario. La estructura varía: la primaria dura 6 o 7 años según la provincia.

La normativa vigente debe verificarse antes de cargar datos, porque hay reformas en curso.

Modelo de datos

El currículo se carga como datos versionados, no como código:

```
Jurisdicción → Nivel → Año/Grado → Área/Materia → Eje → Contenido / NAP
```

Los proyectos y las misiones se vinculan a contenidos con una relación muchos a muchos. Decisiones de diseño:

- **Versionar por jurisdicción y año**, para no romper proyectos viejos.
- **Cargar solo lo del piloto:** una provincia, los años del curso, 3 o 4 áreas (Matemática, Lengua, Ciencias, Sociales).
- **Validación humana** con criterio pedagógico de todo lo que extraiga la IA.
- **Campo de contenido propio** para lo que el docente quiera agregar.

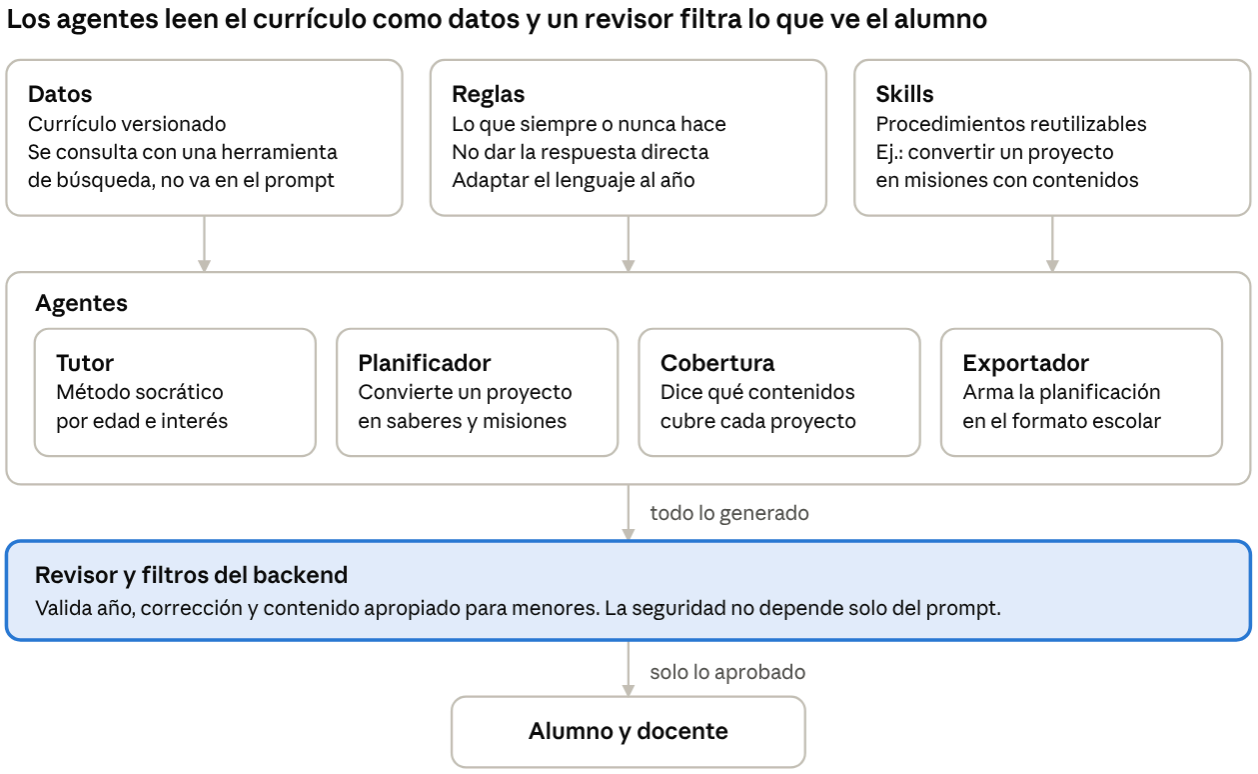
Ventajas frente a la resistencia al cambio

- Tablero de cobertura curricular: qué contenidos oficiales cubre cada proyecto.
- Exportación del proyecto como planificación o secuencia didáctica en el formato de la escuela.
- La app genera evidencia (progreso, portfolio, informes) pero no toca la evaluación oficial.

Arquitectura de agentes

El currículo cargado como datos se usa en los agentes, pero hay que separar tres cosas distintas.

	Qué es	Dónde vive	Ejemplo
Datos	El currículo (contenidos, NAP, años)	Base de datos, consultada por el agente como herramienta	"Dame los contenidos de Ciencias Naturales de 5º año"
Reglas	Lo que el agente siempre o nunca debe hacer	Instrucciones del agente	"Nunca des la respuesta directa"
Skills	Procedimientos paso a paso para una tarea	Archivos de instrucciones reutilizables	"Convertir un proyecto en misiones alineadas a contenidos"



arquitectura de agentes · 3 fuentes, 4 agentes y un revisor

No se pega el currículo entero en el prompt: es caro, se desactualiza y confunde al modelo. El agente lo consulta cuando lo necesita, con una herramienta del estilo `buscar_contenidos(provincia, año, materia)`, y así hay una sola fuente de verdad versionada. Para pasar a otra provincia se cambian los datos; las reglas y los skills se reutilizan.

Los agentes se prueban con casos reales del aula: preguntas raras de los chicos, errores comunes e intentos de que el tutor "dé la respuesta".

Stack técnico y modelo de datos

El stack es una PWA con React y TypeScript, Tailwind y Supabase, el mismo que ya usa el desarrollador en otras apps en producción. Lo nuevo no es la infraestructura sino el dominio y el hecho de manejar datos de menores.

Capa	Decisión	Nota
Frontend	React + TypeScript + Tailwind, PWA	Vite. Next.js solo si aparecen páginas públicas que indexar, y en ese caso como sitio público aparte
Backend y datos	Supabase: Postgres, Auth, RLS, Storage, Edge Functions	Un solo proyecto para todas las escuelas
IA	Llamadas al modelo solo desde el servidor	Con revisor obligatorio y límites por alumno
Hosting	Vercel	Preview y staging con datos ficticios, nunca datos reales de alumnos
Tiendas	Capacitor sobre el mismo código, si hace falta	Se decide después del piloto

Decisión de tenancy. A diferencia del patrón "una base por cliente" que se usa en los ERP, esta plataforma necesita **una sola base con la escuela como unidad de aislamiento** (`escuela_id`), porque los proyectos pueden compartirse entre cursos y escuelas en los niveles altos de la escalera. Cada tabla con datos de alumnos lleva `escuela_id` y su política RLS.

Tablas principales

Grupo	Tablas
Organización	escuelas, membresías (usuario, escuela, rol), cursos, inscripciones
Familias	vínculos familia-alumno, consentimientos (versión del texto, fecha, quién autorizó)
Grupos	grupos (con nombre visible neutro y etiqueta interna solo para el docente), miembros
Currículo	versión de currículo, jurisdicción, área, eje, contenido o NAP

Grupo	Tablas
Proyectos	proyectos (nivel de escalera, estado, visibilidad), contenidos vinculados, misiones, equipos y roles
Perfil	señales del alumno como hipótesis (tipo, valor, actividad de origen, corregida por el docente)
Tutor	sesiones, mensajes mostrados, resultado del revisor
Auditoría	registro de accesos y cambios sensibles

Matriz de acceso por rol

Rol	Ve	Modifica
Alumno	Su propio perfil, misiones, equipo y proyectos	Sus respuestas y su conversación con el tutor
Docente	Alumnos de sus cursos, proyectos y señales como hipótesis	Grupos, proyectos y correcciones de señales
Familia	Solo a su hijo o hija, en formato resumen	Consentimientos
Directivo	Su escuela: agregados, consentimientos pendientes, cobertura curricular	Altas de docentes y cursos
Soporte de la plataforma	Sin acceso a datos de alumnos por defecto	Acceso excepcional, con registro

Reglas fijas para datos de menores

- Toda tabla con datos de alumnos tiene RLS activado y tests automáticos que prueban que cada rol no puede leer lo ajeno.
- Ninguna llamada al modelo de IA sale del cliente; las claves y los prompts viven en el servidor.
- El revisor de contenido es un paso obligatorio entre el agente y el alumno, y queda registrado lo que se le mostró.
- Los entornos de prueba y staging usan datos ficticios.
- Las herramientas de monitoreo no reciben contenido de las conversaciones ni datos personales de alumnos.

- El ingreso de primaria por código de clase vence, es largo y requiere aprobación del docente.

A definir: si el directivo y el docente ven la transcripción completa de las conversaciones con el tutor o solo un resumen, y cómo se crean las sesiones de primaria sin email (por ejemplo, alumnos gestionados por el docente con una Edge Function que valida el código de clase).

Esquema de base de datos

El esquema inicial tiene 26 tablas en cuatro bloques (organización, currículo y proyectos, alumno y tutor, helpers y RLS) y sirve como punto de partida para la primera migración y para la skill `supabase-educacion-multiescuela`. **No fue ejecutado todavía:** hay que correrlo en un Supabase local, revisar las políticas y probarlas con pgTAP antes de usarlo con alumnos reales.

Decisiones de diseño que conviene mantener:

- Todo dato de alumno lleva `escuela_id` y pasa por RLS; sin política no hay acceso.
- Los datos personales se reducen al mínimo: `nombre_visible` y avatar, sin apellido ni email para primaria.
- La etiqueta interna de un grupo vive en una tabla aparte (`grupos_internos`) para que el alumno no pueda leerla.
- Las escrituras sensibles (mensajes del tutor, señales, consentimientos, códigos de clase, solicitudes de ingreso, progreso) las hacen las Edge Functions con `service_role`; el cliente no tiene política de inserción en esas tablas.
- Las señales sobre el alumno nacen como `hipotesis` y solo el docente las confirma, corrige o descarta.
- Cada mensaje del tutor guarda el resultado del revisor, para auditar lo que se le mostró al alumno.

Bloque 1: tipos y organización

```
create type public.rol_escuela as enum
('directivo','docente','alumno','familia');
create type public.nivel_educativo as enum ('primaria','secundaria');
create type public.visibilidad_proyecto as enum
('privado','curso','escuela','publico');
create type public.estado_proyecto as enum
('borrador','activo','finalizado','archivado');

create table public.jurisdicciones (
  id uuid primary key default gen_random_uuid(),
```

```
  nombre text not null unique
);

create table public.escuelas (
  id uuid primary key default gen_random_uuid(),
  jurisdiccion_id uuid not null references public.jurisdicciones(id),
  nombre text not null,
  localidad text,
  activa boolean not null default true,
  created_at timestamptz not null default now()
);

-- Datos mínimos: sin apellido ni email para alumnos de primaria
create table public.perfiles (
  id uuid primary key references auth.users(id) on delete cascade,
  nombre_visible text not null,
  avatar text,
  created_at timestamptz not null default now()
);

create table public.membresias (
  id uuid primary key default gen_random_uuid(),
  escuela_id uuid not null references public.escuelas(id) on delete cascade,
  usuario_id uuid not null references public.perfiles(id) on delete cascade,
  rol public.rol_escuela not null,
  activa boolean not null default true,
  unique (escuela_id, usuario_id, rol)
);

create table public.cursos (
  id uuid primary key default gen_random_uuid(),
  escuela_id uuid not null references public.escuelas(id) on delete cascade,
  nombre text not null,
  nivel public.nivel_educativo not null,
  anio smallint not null,
  ciclo_lectivo smallint not null,
  created_at timestamptz not null default now()
);

create table public.curso_docentes (
  curso_id uuid references public.cursos(id) on delete cascade,
  usuario_id uuid references public.perfiles(id) on delete cascade,
  primary key (curso_id, usuario_id)
);

create table public.inscripciones (
```

```
  curso_id uuid references public.cursos(id) on delete cascade,
  alumno_id uuid references public.perfiles(id) on delete cascade,
  primary key (curso_id, alumno_id)
);

-- Ingreso de primaria por código de clase (se guarda solo el hash)
create table public.codigos_clase (
  id uuid primary key default gen_random_uuid(),
  curso_id uuid not null references public.cursos(id) on delete cascade,
  codigo_hash text not null,
  vence_en timestamptz not null,
  activo boolean not null default true,
  creado_por uuid not null references public.perfiles(id)
);

create table public.solicitudes_ingreso (
  id uuid primary key default gen_random_uuid(),
  curso_id uuid not null references public.cursos(id) on delete cascade,
  nombre_visible text not null,
  estado text not null default 'pendiente' check (estado in
('pendiente','aprobada','rechazada')),
  created_at timestamptz not null default now()
);

create table public.familias_vinculos (
  familia_id uuid references public.perfiles(id) on delete cascade,
  alumno_id uuid references public.perfiles(id) on delete cascade,
  primary key (familia_id, alumno_id)
);

create table public.consentimientos (
  id uuid primary key default gen_random_uuid(),
  escuela_id uuid not null references public.escuelas(id) on delete cascade,
  alumno_id uuid not null references public.perfiles(id) on delete cascade,
  otorgado_por uuid not null references public.perfiles(id),
  version_texto text not null,
  estado text not null check (estado in ('pendiente','otorgado','revocado')),
  fecha timestamptz not null default now()
);

create table public.grupos (
  id uuid primary key default gen_random_uuid(),
  curso_id uuid not null references public.cursos(id) on delete cascade,
  tipo text not null check (tipo in ('formal','dinamico')),
  nombre_visible text not null,   -- lo ven los alumnos, nombre neutro
  criterio jsonb,
```

```
  estado text not null default 'borrador' check (estado in
('borrador','aprobado')),
  creado_por uuid not null references public.perfiles(id)
);

-- Etiqueta que solo ve el docente (por ejemplo 'Refuerzo')
create table public.grupos_internos (
  grupo_id uuid primary key references public.grupos(id) on delete cascade,
  etiqueta text not null
);

create table public.grupo_miembros (
  grupo_id uuid references public.grupos(id) on delete cascade,
  alumno_id uuid references public.perfiles(id) on delete cascade,
  primary key (grupo_id, alumno_id)
);
```

Bloque 2: currículo y proyectos

```
create table public.curriculo_versiones (
  id uuid primary key default gen_random_uuid(),
  jurisdiccion_id uuid not null references public.jurisdicciones(id),
  nombre text not null,
  vigente boolean not null default false,
  validada_por text,   -- persona con criterio pedagógico que validó la carga
  validada_en date,
  created_at timestamptz not null default now()
);

create table public.areas (
  id uuid primary key default gen_random_uuid(),
  version_id uuid not null references public.curriculo_versiones(id) on
delete cascade,
  nombre text not null
);

create table public.curriculo_contenidos (
  id uuid primary key default gen_random_uuid(),
  version_id uuid not null references public.curriculo_versiones(id) on
delete cascade,
  area_id uuid not null references public.areas(id),
  nivel public.nivel_educativo not null,
  anio smallint not null,
  eje text,
  descripcion text not null,
  es_nap boolean not null default false,
```

```
  escuela_id uuid references public.escuelas(id)   -- contenido propio del
docente; null = oficial
);

create table public.proyectos (
  id uuid primary key default gen_random_uuid(),
  escuela_id uuid not null references public.escuelas(id) on delete cascade,
  curso_id uuid references public.cursos(id) on delete set null,
  creado_por uuid not null references public.perfiles(id),
  titulo text not null,
  descripcion text,
  nivel_escalera smallint not null default 1 check (nivel_escalera between 1
and 4),
  estado public.estado_proyecto not null default 'borrador',
  visibilidad public.visibilidad_proyecto not null default 'curso',
  created_at timestamptz not null default now()
);

create table public.proyecto_contenidos (
  proyecto_id uuid references public.proyectos(id) on delete cascade,
  contenido_id uuid references public.curriculo_contenidos(id),
  primary key (proyecto_id, contenido_id)
);

create table public.misiones (
  id uuid primary key default gen_random_uuid(),
  proyecto_id uuid not null references public.proyectos(id) on delete
cascade,
  contenido_id uuid references public.curriculo_contenidos(id),
  orden smallint not null,
  titulo text not null,
  consigna text
);

create table public.proyecto_miembros (
  proyecto_id uuid references public.proyectos(id) on delete cascade,
  alumno_id uuid references public.perfiles(id) on delete cascade,
  rol_equipo text,
  primary key (proyecto_id, alumno_id)
);
```

Bloque 3: alumno y tutor

```
create table public.progreso_misiones (
  mision_id uuid references public.misiones(id) on delete cascade,
  alumno_id uuid references public.perfiles(id) on delete cascade,
  escuela_id uuid not null references public.escuelas(id) on delete cascade,
  estado text not null default 'pendiente' check (estado in
('pendiente','en_curso','completada')),
  pistas_usadas smallint not null default 0,
  completada_en timestamptz,
  primary key (mision_id, alumno_id)
);

create table public.intereses_alumno (
  alumno_id uuid references public.perfiles(id) on delete cascade,
  interes text not null,
  origen text not null check (origen in ('declarado','observado')),
  primary key (alumno_id, interes)
);

-- Observaciones de la app: hipótesis que el docente confirma, corrige o
descarta
create table public.senales_alumno (
  id uuid primary key default gen_random_uuid(),
  escuela_id uuid not null references public.escuelas(id) on delete cascade,
  alumno_id uuid not null references public.perfiles(id) on delete cascade,
  area_id uuid references public.areas(id),
  descripcion text not null,
  mision_origen uuid references public.misiones(id),
  estado text not null default 'hipotesis' check (estado in
('hipotesis','confirmada','corregida','descartada')),
  revisada_por uuid references public.perfiles(id),
  created_at timestamptz not null default now()
);

create table public.tutor_sesiones (
  id uuid primary key default gen_random_uuid(),
  escuela_id uuid not null references public.escuelas(id) on delete cascade,
  alumno_id uuid not null references public.perfiles(id) on delete cascade,
  mision_id uuid references public.misiones(id),
  created_at timestamptz not null default now()
);

create table public.tutor_mensajes (
  id uuid primary key default gen_random_uuid(),
  sesion_id uuid not null references public.tutor_sesiones(id) on delete
cascade,
  escuela_id uuid not null references public.escuelas(id) on delete cascade,
```

```
  autor text not null check (autor in ('alumno','tutor')),
  contenido text not null,   -- lo que efectivamente se mostró
  revision text check (revision in ('aprobado','ajustado','bloqueado')),
  motivo_revision text,
  created_at timestamptz not null default now(),
  check ((autor = 'tutor') = (revision is not null))
);

create table public.audit_log (
  id bigint generated always as identity primary key,
  escuela_id uuid,
  usuario_id uuid,
  accion text not null,
  entidad text,
  entidad_id uuid,
  created_at timestamptz not null default now()
);
```

Bloque 4: helpers, RLS e índices

```
-- Sin acceso anónimo a ninguna tabla
revoke all on all tables in schema public from anon;

-- Helpers: security definer con search_path fijo
create or replace function public.es_miembro(p_escuela uuid, p_rol
public.rol_escuela default null)
returns boolean language sql stable security definer set search_path = '' as
$$
  select exists (
    select 1 from public.membresias m
    where m.usuario_id = (select auth.uid())
      and m.escuela_id = p_escuela
      and m.activa
      and (p_rol is null or m.rol = p_rol)
  );
$$;

create or replace function public.es_docente_del_curso(p_curso uuid)
returns boolean language sql stable security definer set search_path = '' as
$$
  select exists (
    select 1 from public.curso_docentes d
    where d.curso_id = p_curso and d.usuario_id = (select auth.uid())
  );
$$;
```

```
create or replace function public.esta_inscripto(p_curso uuid)
returns boolean language sql stable security definer set search_path = '' as
$$
  select exists (
    select 1 from public.inscripciones i
    where i.curso_id = p_curso and i.alumno_id = (select auth.uid())
  );
$$;

create or replace function public.es_docente_de_alumno(p_alumno uuid)
returns boolean language sql stable security definer set search_path = '' as
$$
  select exists (
    select 1 from public.inscripciones i
    join public.curso_docentes d on d.curso_id = i.curso_id
    where i.alumno_id = p_alumno and d.usuario_id = (select auth.uid())
  );
$$;

create or replace function public.es_familia_de(p_alumno uuid)
returns boolean language sql stable security definer set search_path = '' as
$$
  select exists (
    select 1 from public.familias_vinculos f
    where f.alumno_id = p_alumno and f.familia_id = (select auth.uid())
  );
$$;

create or replace function public.puede_ver_alumno(p_alumno uuid)
returns boolean language sql stable security definer set search_path = '' as
$$
  select p_alumno = (select auth.uid())
      or public.es_docente_de_alumno(p_alumno)
      or public.es_familia_de(p_alumno);
$$;

-- RLS activado en todas las tablas: sin política, no hay acceso
do $$
declare t record;
begin
  for t in select tablename from pg_tables where schemaname = 'public' loop
    execute format('alter table public.%I enable row level security',
t.tablename);
  end loop;
end $$;
```

```
-- Políticas de ejemplo (el resto sigue el mismo patrón y se completa en la
skill)
create policy membresias_select on public.membresias for select to
authenticated
  using (usuario_id = (select auth.uid()) or public.es_miembro(escuela_id,
'directivo'));

create policy perfiles_select on public.perfiles for select to authenticated
  using (id = (select auth.uid()) or public.puede_ver_alumno(id));

create policy grupos_internos_select on public.grupos_internos for select to
authenticated
  using (exists (
    select 1 from public.grupos g
    where g.id = grupo_id and public.es_docente_del_curso(g.curso_id)
  ));

create policy proyectos_select on public.proyectos for select to
authenticated
  using (
    creado_por = (select auth.uid())
    or public.es_docente_del_curso(curso_id)
    or public.esta_inscripto(curso_id)
    or (visibilidad in ('escuela','publico') and
public.es_miembro(escuela_id))
  );

create policy proyectos_insert on public.proyectos for insert to
authenticated
  with check (creado_por = (select auth.uid()) and
public.es_miembro(escuela_id, 'docente'));

create policy senales_select on public.senales_alumno for select to
authenticated
  using (public.es_docente_de_alumno(alumno_id));

create policy senales_update on public.senales_alumno for update to
authenticated
  using (public.es_docente_de_alumno(alumno_id))
  with check (public.es_docente_de_alumno(alumno_id));

-- Ajustar según la decisión pendiente: ¿el docente ve la transcripción
completa?
create policy tutor_mensajes_select on public.tutor_mensajes for select to
authenticated
  using (exists (
```

```
    select 1 from public.tutor_sesiones s
    where s.id = sesion_id
      and (s.alumno_id = (select auth.uid()) or
public.es_docente_de_alumno(s.alumno_id))
  ));

-- Índices sobre las columnas que usan las políticas
create index on public.membresias (usuario_id, escuela_id);
create index on public.inscripciones (alumno_id);
create index on public.curso_docentes (usuario_id);
create index on public.familias_vinculos (alumno_id);
create index on public.proyectos (escuela_id, curso_id);
create index on public.senales_alumno (alumno_id);
create index on public.tutor_sesiones (alumno_id);
create index on public.tutor_mensajes (sesion_id);
```

Pruebas mínimas antes de cualquier piloto

- Una consulta que lista las tablas de `public` sin RLS debe devolver cero filas.
- Cada rol (alumno, docente, familia, directivo) intenta leer datos de otra escuela y de otro curso, y debe recibir cero filas.
- Un alumno intenta leer las señales sobre sí mismo, las etiquetas internas de grupos y los mensajes de otro alumno, y debe fallar.
- Un usuario intenta modificar su propia membresía o su rol, y debe fallar.
- El cliente intenta insertar en `tutor_mensajes`, `senales_alumno`, `consentimientos` y `codigos_clase`, y debe fallar.

Pendientes del esquema: políticas completas de las tablas restantes, políticas de Storage para las planificaciones, vistas con `security_invoker` para los tableros, y el flujo de Edge Function que crea la cuenta de un alumno de primaria al aprobarse la solicitud de ingreso.

Decisiones tomadas

Resumen de lo acordado hasta ahora, para no volver a discutirlo al empezar a construir.

Tema	Decisión	Motivo
Entrada	Por la escuela, con el docente como punto de entrada	Los grupos ya existen y no hay arranque en frío
Arranque	Primaria (5º-6º) primero; secundaria en la fase 2	Un docente de grado cubre casi todas las materias

Tema	Decisión	Motivo
Stack	Vite + React + TypeScript + Tailwind, PWA	Es el que el desarrollador ya domina y usa en producción
Datos	Supabase, una sola base, aislamiento por `escuela_id`	Los proyectos pueden compartirse entre cursos y escuelas
Hosting	Vercel	Preview y staging con datos ficticios
Tiendas	Capacitor sobre el mismo código, después del piloto	No condiciona el arranque
Perfil del alumno	Se construye desde la interacción, no desde documentos subidos	Privacidad y baja fricción
Evaluación	La calificación oficial nunca la genera la app	No invadir la autonomía del docente
Currículo	Datos versionados, una provincia, 3 o 4 áreas	Alcance acotado y validable
Estética	Clara y simple (maqueta de Claude Design), con funciones tomadas de Stitch	Es la dirección que se eligió

Plan de construcción de la fase 1

El orden va de lo que sostiene todo a lo que se ve. Cada paso termina con sus tests y una revisión de seguridad antes de pasar al siguiente.

1. **Base del repositorio:** Vite, React, TypeScript, Tailwind y PWA; CI con tests; despliegue en Vercel; proyectos de Supabase para desarrollo y staging con datos ficticios.
2. **Schema y RLS:** escuelas, membresías, cursos, inscripciones, grupos y consentimientos, con políticas de denegar por defecto y tests por rol.
3. **Ingreso:** cuenta del docente; ingreso de alumnos de primaria por código de clase con aprobación del docente; consentimiento familiar.
4. **Capa curricular:** esquema versionado y carga de una provincia, con validación humana.
5. **Proyectos:** crear proyecto, vincular contenidos, tablero de cobertura curricular, nivel, estado y visibilidad.
6. **Tutor:** Edge Function con las reglas socráticas, revisor obligatorio y registro de lo mostrado; mini-diagnóstico inicial.

7. **Vista del docente:** grupos sugeridos con aprobación, progreso, "lo que vio el tutor" y señales como hipótesis corregibles.
8. **Vista del alumno de primaria:** inicio, misión con voz y proyecto.
9. **Exportar planificación** en el formato de la escuela.
10. **Piloto:** un docente de 5º o 6º, con las métricas definidas más arriba.

Quedan para la fase 2: secundaria, grupos dinámicos, vista de directivo completa, escáner de tareas, co-piloto para familias y matching con financiamiento.

Estructura del repositorio y convenciones

```
src/
  app/            rutas y layouts
  features/       docente, alumno, familia, directivo, proyectos, tutor,
curriculo
  services/       todas las llamadas a Supabase
  components/     componentes compartidos
  lib/            utilidades y tipos
supabase/
  migrations/     migraciones versionadas
  functions/      tutor, revisor, ingreso-por-codigo
  tests/          pgTAP y tests de RLS por rol
docs/             documento de la idea y decisiones
```

- Toda llamada a Supabase pasa por `services/`, nunca directo desde un componente.
- Los tipos de la base se generan desde el esquema, no se escriben a mano.
- Los cambios de base van siempre como migración, nunca directo en producción.
- Ningún secreto en variables con prefijo `VITE_`: ese prefijo las incluye en el código del cliente.
- La lógica del tutor y del revisor vive en Edge Functions, no en el cliente.

Skills del proyecto

Se conserva el método de las skills actuales (un punto de entrada que pregunta antes de cargar, carga diferida, reglas que nunca se omiten) y se reescribe el contenido para este proyecto. Las skills del ERP (clonar proyecto, creativos de Meta, ventas y caja) quedan en el repositorio del ERP.

Skill	Qué cubre	Origen
`arquitecto-edu`	Punto de entrada con flujos por tarea	Reescribir `arquitecto`
`supabase-educacion-multiescuela`	Schema, helpers y RLS por escuela, curso y rol	Nueva
`datos-menores-ia`	Consentimiento, retención, logs, llamadas al modelo	Nueva
`tutor-agentes`	Reglas del tutor, revisor, casos de prueba del aula	Nueva
`curriculo-datos`	Carga y validación del diseño curricular	Nueva
`security-audit-edu`	Las 5 capas actuales más una de menores e IA	Adaptar
`vercel-deploy`	Headers de seguridad, staging ficticio, monitoreo sin datos de alumnos	Adaptar
`diseno-integracion`	Pasar las pantallas diseñadas a React	Adaptar `stitch-integration`

Orden de creación: primero `supabase-educacion-multiescuela`, después `datos-menores-ia`, `tutor-agentes` y recién entonces `arquitecto-edu`, que debe ofrecer skills que ya existan.

Correcciones a tener en cuenta al reutilizar el patrón de comercio: activar RLS en todas las tablas (incluidas perfiles y membresías), fijar `search_path` en las funciones `SECURITY DEFINER`, definir políticas de Storage y usar `security_invoker` en las vistas.

Contexto para el IDE

Texto para pegar como archivo de contexto del proyecto en Visual Studio Code, Antigravity u otro IDE con agentes (el nombre del archivo depende de la herramienta, por ejemplo `CLAUDE.md` o `AGENTS.md`). Se completa y se ajusta con el tiempo.

```
# Contexto del proyecto: [Nombre]

Plataforma educativa con IA para primaria y secundaria en Argentina. Entrada
por la escuela; el docente es el punto de entrada.
```

```
## Qué se construye
Tutor socrático adaptado a los intereses del alumno, grupos que arma el
docente y proyectos interdisciplinarios alineados al currículo oficial (NAP y
diseño curricular provincial).

## Fase actual
Fase 1: núcleo con primaria (5º-6º) y un docente piloto. Secundaria en la
fase 2.

## Stack
Vite + React + TypeScript + Tailwind, PWA (vite-plugin-pwa). Supabase
(Postgres, Auth, RLS, Storage, Edge Functions). Vercel. Una sola base;
aislamiento por escuela_id.

## Roles
alumno, docente, familia, directivo, soporte.

## Reglas que nunca se omiten
- RLS activado en toda tabla, con política de denegar por defecto, y tests
automáticos por rol.
- service_role solo en Edge Functions y tests, nunca en el cliente.
- Ningún secreto en variables VITE_.
- Toda llamada a Supabase pasa por src/services/.
- Ninguna llamada al modelo de IA sale del cliente. Todo lo que genera el
tutor pasa por el revisor antes de llegar al alumno, y se registra lo que se
mostró.
- El tutor no da la respuesta directa: hace preguntas guía, adapta el
lenguaje al año y es neutral en temas políticos.
- La app no genera calificaciones oficiales. Las señales sobre el alumno son
hipótesis que el docente puede corregir; no se etiqueta ni se diagnostica.
- Los entornos de prueba usan datos ficticios. No se envían datos de alumnos
a herramientas de monitoreo.
- Antes de modificar un archivo existente, mostrar el cambio propuesto.

## Frenar y consultar si
- Piden deshabilitar RLS o usar SECURITY DEFINER sin justificación.
- Aparece una clave o token en el código.
- Se registra contenido de conversaciones de alumnos en logs.
- Hay un push a main sin tests pasando.

## Más contexto
Ver el documento de la idea del proyecto.
```

Checklist de arranque

- [ ] Confirmar la provincia del piloto y reunir los documentos curriculares oficiales.
- [ ] Crear el repositorio y el proyecto en Vercel.
- [ ] Crear dos proyectos de Supabase (desarrollo y staging) con datos ficticios.
- [ ] Guardar el archivo de contexto en la raíz del repositorio.
- [ ] Escribir y probar la skill `supabase-educacion-multiescuela`.
- [ ] Definir la política de consentimiento familiar y de datos de menores con asesoría legal.
- [ ] Elegir al docente de primaria para el piloto.
- [ ] Guardar las maquetas finales en `docs/` como referencia visual.

Estrategia por nivel: primaria y secundaria

Se construye **un solo núcleo técnico con dos modos de experiencia**: la base (grupos, proyectos, tutor, portfolio) es común y cambia la capa de interfaz y de reglas según la edad. Hay acceso a escuelas y docentes de ambos niveles.

	Primaria (arranque: 5º-6º)	Secundaria (arranque: 3º-4º año)
Quién usa la app	Docente y familia median casi todo	El alumno es autónomo
Tutor	Más guiado, juego, personajes, textos cortos, voz	Socrático profundo, pide argumentar
Grupos	Los arma el docente	Dinámicos, por interés o proyecto
Proyectos	De aula, tipo huerta o feria de ciencias	Con impacto real, prototipos, emprendimiento
Financiamiento y contacto externo	Ninguno	Sí, siempre mediado por docente o institución
Privacidad	Más estricta, consentimiento familiar obligatorio	Consentimiento familiar, más margen de autonomía

Por qué primaria primero. Un docente de grado dicta casi todas las materias, así que con un solo docente convencido ya hay un grupo completo y un proyecto interdisciplinario. En secundaria cada curso tiene 8 o 10 profesores con horas sueltas, y coordinar un proyecto integrador exige convencer a varios.

Riesgos de primaria: pocos chicos tienen celular propio (el uso sería en el aula o en casa con la familia), la interfaz necesita voz e imágenes, y los filtros de contenido deben ser más fuertes.

Cómo no perder los proyectos escalables: cada proyecto lleva desde el día uno un nivel, un estado y una visibilidad (privado, de la escuela, público). En primaria se usan los primeros niveles; en secundaria se activan los niveles 3 y 4.

Modelo de interacción y perfil del alumno

La app es la misma en ambos niveles, pero la autonomía crece con la edad. Regla general: **el docente controla lo estructural** (grupos, proyectos, contenidos) y **el alumno controla el cómo** (interés, ritmo, forma de explorar). La entrada es por la escuela.

	Primaria (5º-6º)	Secundaria (3º-4º año)
Ingreso	Código de clase o QR, sin usuario ni contraseña propios	Cuenta propia, vinculada a la escuela
Dónde se usa	Sobre todo en el aula, y en casa con un adulto cerca	En el aula y por su cuenta, en el celular
Tutor	Guiado: misiones cortas, voz, opciones para elegir, texto libre acotado	Conversación más abierta, con preguntas que piden argumentar
Intereses	Los elige de un menú	Los declara y la app los va afinando
Proyectos	Los crea y asigna el docente	El alumno propone, el docente aprueba
Grupos	Los arma el docente	Se forman por proyecto o afinidad, con aprobación
Docente	Conduce la sesión y ve todo	Supervisa, acompaña y ve el progreso
Familia	Resumen y sugerencias	Consentimiento y resumen general

En secundaria el alumno debe saber qué ve el docente de su actividad, para que la supervisión sea transparente.

Cómo se construye el perfil del alumno

El perfil se arma principalmente desde la interacción con la app, no desde documentos subidos. Entrando por la escuela no se puede depender de que las familias carguen archivos, y evitar guardar evaluaciones e informes reduce el riesgo de privacidad con datos sensibles de menores.

- **Nivel:** mini-diagnóstico lúdico al empezar (5 a 8 minutos) y ajuste continuo por contenido, vinculado a los NAP.
- **Intereses:** declarados al inicio y observados después (con qué analogías sigue enganchado).
- **Dificultades:** errores recurrentes, en qué paso pide ayuda, qué conceptos revisita.
- **Motivación:** constancia, tipo de misión que completa, cuándo abandona.

Estas señales se muestran al docente como **hipótesis que puede corregir**. La app no etiqueta ni diagnostica: no dice "tiene dislexia", a lo sumo "se traba con consignas largas".

Qué documentos se pueden subir

- **Docente:** su planificación y las guías de ejercicios. Es el contexto más valioso y menos sensible, y mantiene todo alineado al currículo.
- **Alumno o familia:** foto de un ejercicio puntual para pedir ayuda, sin guardarla como archivo de perfil.
- **Evaluaciones pasadas e informes:** fuera del MVP. Si se suman después, opcionales, con consentimiento explícito y mediados por la escuela.

Los criterios de privacidad conviene validarlos con asesoría legal bajo la Ley 25.326.

MVP y alcance del piloto

El MVP valida un flujo central: **el docente crea un grupo, define un proyecto simple, la app genera misiones adaptadas por interés y el docente ve el progreso.**

Entra en el MVP

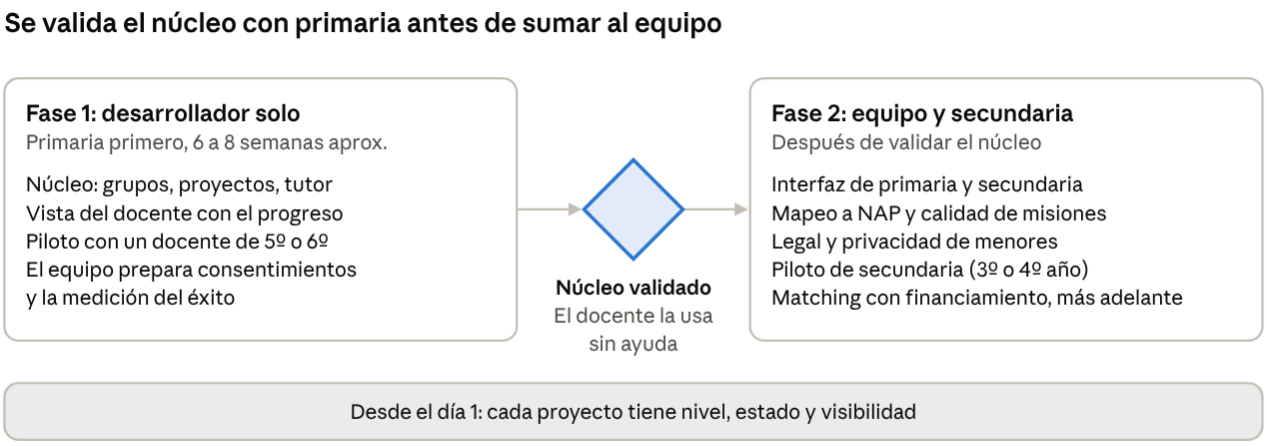
- Grupos formales creados por el docente.
- Proyectos simples con nivel, estado y visibilidad en el modelo de datos.
- Capa curricular de una provincia, de los años del piloto, 3 o 4 áreas.
- Tutor socrático con personalización por interés.
- Vista del docente: progreso y dónde se traba cada alumno.
- Consentimiento familiar y filtros de seguridad para menores.

Queda afuera por ahora

- Grupos dinámicos sugeridos por la app.
- Matching con financiamiento, mentores y empresas.
- Escáner de tareas y co-piloto para padres.
- Otras provincias.

Plan por fases

El desarrollo va en dos fases: primero el desarrollador solo con primaria y, con el núcleo validado, se suma el equipo y arranca secundaria.



plan por fases · 2 fases y 1 puerta de validación

Los pilotos van en secuencia y no en paralelo porque el desarrollo inicial lo hace una sola persona. Durante la fase 1, el equipo se ocupa de lo que no es código y no frena el desarrollo: conseguir el consentimiento de las familias y diseñar la medición del éxito, para que cuando el núcleo esté listo el piloto ya tenga todo preparado.

Cada piloto arranca con **un solo docente por nivel**, no con cinco: mejor pocos que la usen de verdad. La duración de la fase 1 es una estimación inicial a confirmar.

Métricas de éxito del piloto

Se definen antes de arrancar. Los umbrales son una propuesta inicial para ajustar con el equipo.

Métrica	Qué valida	Umbral propuesto
El docente crea un proyecto sin ayuda del desarrollador	Que el flujo es autónomo	1 proyecto completo sin asistencia
Alumnos que completan al menos una misión	Que el tutor engancha	70%
Docente que sigue usando la app tras la 3ª semana	Adopción real	Uso semanal sostenido
Cobertura curricular de los proyectos	Que no se sale del currículo	Contenidos oficiales vinculados en cada proyecto

Métrica	Qué valida	Umbral propuesto
Proyectos de secundaria que llegan al nivel 3	Que la escalera funciona	Al menos 1

Modelo de financiamiento

La app probablemente combine más de una fuente. Todas son hipótesis a validar con las escuelas.

Fuente	Cómo funciona
Escuelas privadas	Licencia institucional
Familias	Modelo freemium
Gobiernos provinciales o municipales	Contratos o programas piloto
Fundaciones y empresas	Financian el acceso para escuelas públicas

La conexión de proyectos con financiamiento externo (ferias, concursos, fundaciones, programas de emprendimiento joven) es una segunda línea de valor, recién desde secundaria.

Riesgos y cuidados

- **Privacidad de menores:** consentimiento familiar y manejo de datos conforme a la Ley 25.326 de protección de datos personales. Se resuelve antes del piloto, no después. Conviene revisarlo con asesoría legal.
- **Seguridad del tutor:** las reglas con menores no dependen solo del prompt; se refuerzan con filtros y moderación en el backend.
- **Errores en el currículo cargado:** si la base tiene un error, todos los agentes lo heredan. Validación humana obligatoria.
- **Contacto externo en secundaria:** los niveles 3 y 4 siempre con mediación docente o institucional, para proteger a los menores del contacto con adultos externos.
- **Propiedad intelectual:** definir de quién es un proyecto creado por menores antes de conectarlo con financiadores.
- **Resistencia al cambio:** mitigada con la capa curricular, la exportación de planificaciones y no tocar la evaluación oficial.
- **Dispersión del alcance:** tutor, grupos, proyectos y financiamiento es mucho; por eso el MVP es acotado y por fases.

Preguntas abiertas y próximos pasos

- [ ] ¿En qué provincia están las escuelas del piloto? Define qué diseño curricular se carga.
- [ ] ¿Qué docente de primaria (5º o 6º) y cuál de secundaria (3º o 4º año) serán el primer piloto?
- [ ] Buscar y verificar los documentos curriculares oficiales y la normativa vigente de esa provincia.
- [ ] Diseñar el esquema de la capa curricular y cargar 3 o 4 áreas.
- [ ] Definir el consentimiento familiar y la política de datos de menores.
- [ ] Acordar con el equipo qué tareas toman durante la fase 1 (consentimientos, medición del éxito).
- [ ] Confirmar las métricas y los umbrales del piloto.